

Learning-Rate Scaling & Training Dynamics

What breaks when your effective batch size is 2048 instead of 32 — and how to keep training stable at any scale.

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

The Big Picture — Why This Session Exists

When you move a training job from 1 GPU to 8 or 64, you don't just get more speed for free. Three things quietly change underneath you.

1

Effective batch size jumps

More GPUs in parallel means each 'step' processes far more examples at once, whether you planned for it or not.

2

Learning rate stops matching

A learning rate tuned for batch=32 is quietly wrong at batch=2048 — usually meaning slower convergence or outright divergence.

3

New failure modes appear

Stragglers and loss spikes only show up at scale — they're invisible on a single GPU.

THINK OF IT LIKE THIS: Doubling a cake recipe isn't just 'use twice the flour.' Bake time and mixing time change too — otherwise you get a burnt outside and a raw middle. Scaling a training job works the same way: batch size is the ingredient everyone remembers to scale, but learning rate, warmup, and stability need to scale WITH it.

Key Terms Before We Start

A quick shared vocabulary — skip ahead if these are all second nature to you.

Batch size — How many training examples the model looks at before updating its weights once.

Learning rate (LR) — How big a step the optimizer takes when it updates the weights.

Gradient — The direction and size of the 'nudge' each weight should get, from the current batch.

Step / iteration — One cycle of: look at a batch, compute gradients, update weights.

Epoch — One full pass through the entire training dataset.

Convergence / divergence — Loss steadily settling down, vs. loss exploding or oscillating forever.

Why LR Must Scale With Batch Size

SGD takes a step: $w \leftarrow w - lr * grad$. The gradient (grad) is the AVERAGE of per-example gradients across the batch.

THINK OF IT LIKE THIS: Ask ONE person which way the wind is blowing and you get a shaky, unreliable answer. Ask 1,000 people and average their answers, and you get a steady, trustworthy signal — now you can act on it confidently. A bigger batch is like asking more people: the averaged gradient is less noisy, so it's SAFE to take a bigger step.

Linear scaling rule (Goyal et al. 2017)

$lr_new = lr_base * (batch_new / batch_base)$
Example: $lr=0.1$ at $batch=256$. At $batch=2048$: $lr = 0.1 * 8 = 0.8$.

Effective batch in DDP

$effective_batch = per_gpu_batch * world_size * grad_accum_steps$. 1 GPU to 8 = 8x bigger batch before you change any code.

THE TRAP: distributed training silently multiplies your batch size. Forget to scale LR and you're training with an LR that's implicitly too small for the noise level of your gradients.

Worked Example — Doing the Math Slowly

Let's walk through a realistic scenario, one step at a time.

```
SCENARIO: tuned on 1 GPU, batch=32, lr=1e-4 works well.  
Now moving to 4 nodes x 8 GPUs = 32 GPUs, same per-GPU batch of 32.  
  
STEP 1 — new effective batch:  32 * 32 = 1024  
STEP 2 — scale factor:  1024 / 32 = 32  
STEP 3 — new LR:  1e-4 * 32 = 3.2e-3  
STEP 4 — add warmup (next section): ramp lr 0 -> 3.2e-3 over 1000-2000 steps
```

A 32x jump in GPUs means a 32x jump in effective batch, which means a 32x jump in LR. That's a BIG change — exactly why we can't skip warmup.

Three Scaling Rules — Which Applies When

Linear

$$lr = lr_base * (B/B_base)$$

USE: SGD+momentum, most vision, up to ~8192.

LIMIT: breaks above ~32K for most CNNs.

Square-root

$$lr = lr_base * \sqrt{B/B_base}$$

USE: Adam family.

LIMIT: conservative, often underscales.

Adaptive (LAMB/LARS)

Per-layer trust ratio, no single global LR.

USE: 32K+ batches.

LIMIT: extra tuning complexity.

DEFAULT: linear. Scaling to 32K+ batches (rare outside frontier pretraining)? Investigate LAMB.

Quick Decision Guide — Which Rule Do I Pick?

Your situation	Pick this rule
Default / not sure / batch under 8K	Linear — start here
Using Adam and linear feels unstable	Square-root as a fallback
Batch is 32K or larger (frontier pretraining)	LAMB / LARS
Fine-tuning, small batch changes	Linear, often barely needed

You will almost never leave the first row. This exists so you recognise the RARE cases where you should.

Warmup and Schedules

First iterations have garbage (randomly initialised) weights. Full LR + noisy gradients here = divergence.

THINK OF IT LIKE THIS: You don't floor the accelerator the instant you start a cold engine. Warmup does the same for training: linearly ramp LR from near-zero to full value over the first steps, giving the model time to find its footing.

Defaults that work

LLM pretrain: 500-2000 step warmup, cosine decay.

Fine-tune: constant/linear warmup + decay.

Vision: 5-epoch warmup, step decay.

Why Warmup Works — the Intuition

PICTURE THIS: Finding the lowest point in a hilly landscape while blindfolded, feeling the slope with your feet. At the start you have NO idea which way is downhill — the terrain (randomly initialised weights) is chaotic. Giant leaps immediately are as likely to go off a cliff as toward the valley. Start with small steps to feel the terrain, then take bigger strides once you're on sensible ground.

Without warmup

Full-size steps hit chaotic weights. Gradients are especially unreliable here. Result: loss explodes or gets stuck within ~100 steps.

With warmup

Small early steps let the model settle. Gradients become more trustworthy as training progresses. By full LR, the model can handle it.

Stragglers — When One Machine Slows Everyone

All GPUs must finish a step before anyone moves on. One slow GPU means EVERYONE waits.

THINK OF IT LIKE THIS: A hiking group that must regroup at each checkpoint. One hiker with a pebble in their shoe drops the WHOLE group's pace to their pace.

How to spot one

Symptom: N-GPU training barely beats 1 GPU.

Diagnostic: log step time PER RANK, not just average.

Cause: bad GPU, noisy neighbour, slow storage mount, dusty cable.

What to do

Confirm the pattern for 50+ steps.

SSH the suspect node, check for thermal throttling.

Drain the node and relaunch on healthy ones.

Loss Spikes — When Training Suddenly Blows Up

Loss looks healthy, then suddenly jumps 10-100x out of nowhere.

Common causes

LR too high (often right as warmup ends).
fp16 overflow — switch to bf16.
An unusual/corrupted data batch.
General numerical instability.

How to recover

Gradient clipping (`max_norm=1.0`).
Skip a step if grad norm is many std devs above normal.
Roll back to the last checkpoint and resume.

REAL-WORLD: large-scale teams (e.g. OPT-175B) treat 'spike-and-resume' as routine, not an emergency — checkpoint often, roll back, skip a batch, resume.

Why Linear Scaling Eventually Breaks

THINK OF IT LIKE THIS: Adding a second cook roughly doubles vegetable-prep speed. A tenth cook barely helps — elbows bump, counter space runs out. Batch size is the same: past a 'critical batch size', more data per step stops buying a proportionally better gradient, and scaling LR further no longer helps.

```
# B_crit ~ trace(Cov(gradient)) / |E[gradient]|^2    (McCandlish et al. 2018)
# Below B_crit: doubling batch ~halves steps needed -> linear scaling holds
# Above B_crit: doubling batch barely helps -> scaling gains diminish
```

TAKEAWAY: diminishing returns from scaling batch further? You may be past B_{crit} — a different optimizer (LAMB), not just more warmup.

Common Mistakes & How to Spot Them

Mistake	How it shows up	Fix
Scaling GPUs, not LR	Slower convergence than single-GPU, no obvious cause	Recompute LR with the linear scaling rule
Skipping warmup on scale-up	Loss diverges/NaNs in first 100-500 steps	Add 500-2000 steps of linear warmup
Only watching avg throughput	Cluster 'feels slow', no obvious cause	Log step time PER RANK
Treating a spike as fatal	Team panics, restarts from scratch	Roll back to checkpoint, skip a batch, resume

Quick Recap — Terms and Formulas

Linear scaling rule — $lr_new = lr_base * (batch_new / batch_base)$

Effective batch (DDP) — $per_gpu_batch * world_size * grad_accum_steps$

Warmup — Ramp LR from ~0 to full value over the first 500-2000 steps

Straggler — One slow worker that bottlenecks the entire distributed job

Critical batch size (B_crit) — The point past which bigger batches stop helping proportionally

Session 1 Summary

- Larger batches average over more examples, reducing gradient noise — safe to take bigger LR steps
- Linear scaling rule (default) for SGD; sqrt for Adam; LAMB/LARS for 32K+ batches
- Warmup is non-optional at scale — ramp LR over 500-2000 steps to avoid early divergence
- Straggler signature: one rank consistently slower — diagnose per-rank, drain the bad node
- Loss spikes: usually LR too high, fp16 overflow, or bad data — clip gradients, checkpoint-and-resume

NEXT: Checkpointing & Fault Tolerance — sharded checkpoints, elastic training.